

# Effective C++ 第三版 55 条款总结

来源文件：Effective C++ 中文版第三版.pdf

范围：只总结书中 55 个条款

## 目录

- 条款01：视 C++ 为一个语言联邦
- 条款02：尽量以 `const`、`enum`、`inline` 替换 `#define`
- 条款03：尽可能使用 `const`
- 条款04：确定对象被使用前已先被初始化
- 条款05：了解 C++ 默默编写并调用哪些函数
- 条款06：若不想使用编译器自动生成的函数，就该明确拒绝
- 条款07：为多态基类声明 `virtual` 析构函数
- 条款08：别让异常逃离析构函数
- 条款09：绝不在构造和析构过程中调用 `virtual` 函数
- 条款10：令 `operator=` 返回一个 `reference to *this`
- 条款11：在 `operator=` 中处理自我赋值
- 条款12：复制对象时勿忘其每一个成分
- 条款13：以对象管理资源
- 条款14：在资源管理类中小心 `copying` 行为
- 条款15：在资源管理类中提供对原始资源的访问
- 条款16：成对使用 `new` 和 `delete` 时要采取相同形式
- 条款17：以独立语句将 `newed` 对象置入智能指针
- 条款18：让接口容易被正确使用，不易被误用
- 条款19：设计 `class` 犹如设计 `type`
- 条款20：宁以 `pass-by-reference-to-const` 替换 `pass-by-value`
- 条款21：必须返回对象时，别妄想返回其 `reference`
- 条款22：将成员变量声明为 `private`
- 条款23：宁以 `non-member`、`non-friend` 替换 `member` 函数
- 条款24：若所有参数皆需类型转换，请为此采用 `non-member` 函数
- 条款25：考虑写出一个不抛异常的 `swap` 函数
- 条款26：尽可能延后变量定义式的出现时间

- 条款27：尽量少做转型
- 条款28：避免返回 handles 指向对象内部成分
- 条款29：为异常安全而努力是值得的
- 条款30：透彻了解 inlining 的里里外外
- 条款31：将文件间的编译依存关系降至最低
- 条款32：确定你的 public 继承塑模出 is-a 关系
- 条款33：避免遮掩继承而来的名称
- 条款34：区分接口继承和实现继承
- 条款35：考虑 virtual 函数以外的其他选择
- 条款36：绝不重新定义继承而来的 non-virtual 函数
- 条款37：绝不重新定义继承而来的缺省参数值
- 条款38：通过复合塑模出 has-a 或根据某物实现出
- 条款39：明智而审慎地使用 private 继承
- 条款40：明智而审慎地使用多重继承
- 条款41：了解隐式接口和编译期多态
- 条款42：了解 typename 的双重意义
- 条款43：学习处理模板化基类内的名称
- 条款44：将与参数无关的代码抽离 templates
- 条款45：运用成员函数模板接受所有兼容类型
- 条款46：需要类型转换时请为模板定义非成员函数
- 条款47：请使用 traits classes 表现类型信息
- 条款48：认识 template 元编程
- 条款49：了解 new-handler 的行为
- 条款50：了解 new 和 delete 的合理替换时机
- 条款51：编写 new 和 delete 时需固守常规
- 条款52：写了 placement new 也要写 placement delete
- 条款53：不要轻忽编译器的警告
- 条款54：让自己熟悉包括 TR1 在内的标准程序库
- 条款55：让自己熟悉 Boost

---

## 条款01：视 C++ 为一个语言联邦

C++ 不是单一风格的语言，而是由 C、面向对象 C++、模板 C++、STL 四个主要子语言组成。写代码时要根据所处子语言选择规则：C 部分重视低级对

象和指针，面向对象部分重视封装、继承、多态，模板部分重视编译期抽象，STL 部分重视迭代器、算法和容器之间的约定。

例如，在普通对象传参时常用 `const T&` 避免拷贝；但在 STL 迭代器、函数对象等小型对象上，按值传递反而常见，因为它们本来就被设计成轻量对象。

```
#include <algorithm>
#include <vector>

class BigImage {
public:
    void draw() const {}
};

void render(const BigImage& image) { // 面向对象场景：避免复制大对象
    image.draw();
}

int main() {
    std::vector<int> v{3, 1, 2};
    std::sort(v.begin(), v.end()); // STL 场景：迭代器按值传递很自然
}
```

因此不要机械套用单条经验。面对类层次、模板、STL 容器时，判断标准会变化。

## 条款02：尽量以 `const`、`enum`、`inline` 替换 `#define`

`#define` 由预处理器处理，不进入普通 C++ 作用域和类型系统，调试时也可能看不到符号名。常量优先使用 `const` 或 `enum`，函数宏优先使用 `inline` 函数或模板函数。

```
#define ASPECT_RATIO 1.653 // 不推荐：没有类型，也不受作用域限制

const double AspectRatio = 1.653; // 推荐

class GamePlayer {
private:
    static const int NumTurns = 5; // 类内常量
    int scores[NumTurns];
};

template <typename T>
inline const T& maxValue(const T& a, const T& b) { // 替代宏函数
    return a < b ? b : a;
}
```

函数宏容易重复求值，导致副作用被执行多次。

```
#define CALL_WITH_MAX(a, b) f((a) > (b) ? (a) : (b))

int a = 5, b = 0;
// CALL_WITH_MAX(++a, b); 可能让 ++a 执行两次

inline void f(int) {}
CALL_WITH_MAX(++a, b); // 危险
```

使用 `inline` 模板函数能保留类型检查、作用域、调试信息，并避免重复求值。

## 条款03：尽可能使用 `const`

`const` 能把“不应改变”的意图交给编译器检查。它可以修饰变量、指针、函数参数、返回值和成员函数。对成员函数加 `const` 表示该函数不会改变对象的逻辑状态，因此 `const` 对象也能调用它。

```

class TextBlock {
public:
    TextBlock(std::string text) : text_(std::move(text)) {}

    char operator[](std::size_t pos) const { // const 对象可调用
        return text_[pos];
    }

    char& operator[](std::size_t pos) { // 非 const 对象可修改
        return text_[pos];
    }

private:
    std::string text_;
};

void print(const TextBlock& tb) {
    char c = tb[0]; // 调用 const 版本
    // tb[0] = 'x'; // 错误: const 版本返回 char, 不可赋值
}

```

当 `const` 和非 `const` 成员函数逻辑相同, 推荐让非 `const` 版本调用 `const` 版本, 避免重复实现。

```

class Buffer {
public:
    const char& at(std::size_t i) const {
        return data_[i];
    }

    char& at(std::size_t i) {
        return const_cast<char&>(
            static_cast<const Buffer&>(*this).at(i)
        );
    }

private:
    std::string data_ = "abc";
};

```

## 条款04：确定对象被使用前已先被初始化

内置类型在某些场景下不会自动初始化，类成员初始化顺序也不是初始化列表的书写顺序，而是成员声明顺序。构造函数应使用成员初始化列表，而不是先默认构造再赋值。

```
class PhoneNumber {};  
  
class ABEntry {  
public:  
    ABEntry(const std::string& name, const std::string& address,  
            const PhoneNumber& phone)  
        : name_(name),  
          address_(address),  
          phone_(phone),  
          numTimesConsulted_(0) {}  
  
private:  
    std::string name_;  
    std::string address_;  
    PhoneNumber phone_;  
    int numTimesConsulted_;  
};
```

还要注意跨编译单元的非局部静态对象初始化顺序不确定。可用“函数内局部静态对象”避免顺序问题。

```

class FileSystem {
public:
    std::size_t numDisks() const { return 1; }
};

FileSystem& fileSystem() {
    static FileSystem fs; // 第一次调用时初始化
    return fs;
}

class Directory {
public:
    Directory() {
        disks_ = fileSystem().numDisks();
    }
private:
    std::size_t disks_ = 0;
};

```

## 条款05：了解 C++ 默默编写并调用哪些函数

如果你没有声明，编译器可能为类生成默认构造函数、析构函数、拷贝构造函数、拷贝赋值运算符。它们只做成员级操作：逐个构造、逐个析构、逐个拷贝或赋值。

```

class Empty {};

// 编译器概念上可能生成：
// Empty::Empty();
// Empty::~~Empty();
// Empty::Empty(const Empty&);
// Empty& Empty::operator=(const Empty&);

```

但生成并不代表总能用。例如类里有引用成员或 `const` 成员，默认赋值运算符往往无法生成。

```

class NamedObject {
public:
    NamedObject(std::string& name, const int id)
        : nameValue_(name), objectId_(id) {}

private:
    std::string& nameValue_;
    const int objectId_;
};

// NamedObject a(...), b(...);
// a = b; // 错误：引用成员不能改绑，const 成员不能赋值

```

一旦类拥有资源、继承关系或不可复制语义，就不要依赖默认生成函数，要明确写出或禁止。

## 条款06：若不想使用编译器自动生成的函数，就该明确拒绝

有些对象天生不应复制，例如互斥锁、文件句柄拥有者、数据库连接。若不禁止，编译器可能生成拷贝操作，导致多个对象同时认为自己拥有同一资源。

C++11 之后推荐用 `= delete` 明确拒绝。

```

class HomeForSale {
public:
    HomeForSale() = default;

    HomeForSale(const HomeForSale&) = delete;
    HomeForSale& operator=(const HomeForSale&) = delete;
};

int main() {
    HomeForSale h1;
    // HomeForSale h2(h1); // 编译错误
}

```



在旧式 C++ 中常把拷贝构造和赋值运算符声明为 `private` 且不实现。现代代码直接用 `delete` 更清楚，错误也更早暴露。

## 条款07：为多态基类声明 `virtual` 析构函数

如果一个类打算作为多态基类使用，也就是有 `virtual` 函数并允许通过基类指针删除派生对象，那么析构函数必须是 `virtual`。否则 `delete basePtr` 只调用基类析构函数，派生类资源无法释放，行为未定义。

```
class Shape {
public:
    virtual ~Shape() = default;
    virtual void draw() const = 0;
};

class Circle : public Shape {
public:
    ~Circle() override {
        // 释放 Circle 自己管理的资源
    }

    void draw() const override {}
};

void destroy(Shape* p) {
    delete p; // 正确：会调用 Circle::~~Circle 再调用 Shape::~~Shape
}
```

如果类不作为基类使用，或不需要多态，不要随意加 `virtual` 析构函数，因为它会引入虚表指针等额外成本。

## 条款08：别让异常逃离析构函数

析构函数常在栈展开期间被调用。如果析构函数再抛出异常，程序可能直接终止。析构函数内部应捕获异常，记录日志、吞掉异常，或提供普通成员函数让用户显式执行可能失败的操作。

```

class DBConnection {
public:
    void close() {
        // 可能抛异常
    }
};

class DBConn {
public:
    void close() {
        db_.close();
        closed_ = true;
    }

    ~DBConn() {
        if (!closed_) {
            try {
                db_.close();
            } catch (...) {
                // 记录日志，不能让异常离开析构函数
            }
        }
    }

private:
    DBConnection db_;
    bool closed_ = false;
};

```

如果关闭失败对业务很重要，应让用户主动调用 `close()` 并处理异常；析构函数只做兜底清理。

## 条款09：绝不在构造和析构过程中调用 virtual 函数

构造基类时，派生类部分还没有构造好；析构基类时，派生类部分已经析构完。此时调用 `virtual` 函数不会分派到派生类版本，而是调用当前构造/析构阶段所属类的版本。

```

class Transaction {
public:
    Transaction() {
        // logTransaction(); // 不推荐：构造期间调用 virtual
    }

    virtual void logTransaction() const = 0;
    virtual ~Transaction() = default;
};

class BuyTransaction : public Transaction {
public:
    void logTransaction() const override {}
};

```

更好的做法是把需要的信息作为构造参数传给基类的非虚函数。

```

class Transaction2 {
public:
    explicit Transaction2(const std::string& logInfo) {
        log(logInfo);
    }

private:
    void log(const std::string& info) {}
};

class BuyTransaction2 : public Transaction2 {
public:
    BuyTransaction2() : Transaction2(createLogString()) {}

private:
    static std::string createLogString() {
        return "buy transaction";
    }
};

```

## 条款10：令 operator= 返回一个 reference to \*this

赋值运算符应返回左侧对象的引用，即 `*this`。这样才能支持连续赋值，也符合内置类型的行为。

```
class Widget {
public:
    Widget& operator=(const Widget& rhs) {
        if (this != &rhs) {
            value_ = rhs.value_;
        }
        return *this;
    }

private:
    int value_ = 0;
};

int main() {
    Widget a, b, c;
    a = b = c; // b = c 返回 b，随后 a = b
}
```

这个约定也适用于 `+=`、`-=`、`*=` 等赋值相关运算符。

```
class Counter {
public:
    Counter& operator+=(int n) {
        value_ += n;
        return *this;
    }

private:
    int value_ = 0;
};
```

## 条款11：在 operator= 中处理自我赋值

自我赋值可能来自明显写法 `a = a`，也可能来自别名、引用、指针间接造成。赋值运算符必须在自我赋值时仍然安全。

```
class Bitmap {};  
  
class Widget {  
public:  
    Widget() : pb_(new Bitmap) {}  
  
    Widget(const Widget& rhs)  
        : pb_(new Bitmap(*rhs.pb_)) {}  
  
    Widget& operator=(const Widget& rhs) {  
        if (this == &rhs) {  
            return *this;  
        }  
  
        delete pb_;  
        pb_ = new Bitmap(*rhs.pb_);  
        return *this;  
    }  
  
    ~Widget() {  
        delete pb_;  
    }  
  
private:  
    Bitmap* pb_;  
};
```

上面虽然处理了自我赋值，但异常安全不够：`new` 失败时对象已经丢失旧资源。更好的写法是先复制，再替换。

```

class SafeWidget {
public:
    SafeWidget() : pb_(new Bitmap) {}

    SafeWidget(const SafeWidget& rhs)
        : pb_(new Bitmap(*rhs.pb_)) {}

    SafeWidget& operator=(const SafeWidget& rhs) {
        Bitmap* newBitmap = new Bitmap(*rhs.pb_);
        delete pb_;
        pb_ = newBitmap;
        return *this;
    }

    ~SafeWidget() {
        delete pb_;
    }

private:
    Bitmap* pb_;
};

```

现代 C++ 中还可使用 copy-and-swap，把自我赋值和异常安全一起处理。

## 条款12：复制对象时勿忘其每一个成分

手写拷贝构造函数或拷贝赋值运算符时，必须复制对象的所有成员，也必须调用基类的相应拷贝函数。新增成员后要同步更新复制逻辑。

```

class Customer {
public:
    Customer(const Customer& rhs)
        : name_(rhs.name_) {}

    Customer& operator=(const Customer& rhs) {
        name_ = rhs.name_;
        return *this;
    }

private:
    std::string name_;
};

class PriorityCustomer : public Customer {
public:
    PriorityCustomer(const PriorityCustomer& rhs)
        : Customer(rhs),
          priority_(rhs.priority_) {}

    PriorityCustomer& operator=(const PriorityCustomer& rhs) {
        Customer::operator=(rhs);
        priority_ = rhs.priority_;
        return *this;
    }

private:
    int priority_ = 0;
};

```

不要让拷贝构造函数调用拷贝赋值运算符，或反过来调用。它们一个负责初始化新对象，一个负责修改已存在对象，语义不同。可把公共逻辑提取到普通私有函数。

## 条款13：以对象管理资源

资源包括堆内存、文件描述符、互斥锁、数据库连接等。获取资源后应立即放进对象，让析构函数负责释放，这就是 RAII。这样无论函数正常返回还是异常退出，资源都会被释放。

```
#include <memory>

class Investment {
public:
    virtual ~Investment() = default;
};

Investment* createInvestment();

void f() {
    std::unique_ptr<Investment> pInv(createInvestment());
    // 中途即使抛异常，unique_ptr 也会 delete 资源
}
```

不要裸 `new` 后依赖手写 `delete`。

```
void bad() {
    Investment* p = createInvestment();
    // doSomething(); 如果这里抛异常，delete p 不会执行
    delete p;
}
```

现代 C++ 优先使用 `std::unique_ptr` 表示独占所有权，使用 `std::shared_ptr` 表示共享所有权。

## 条款14：在资源管理类中小心 copying 行为

RAII 类被复制时必须明确语义。常见选择有四种：禁止复制、引用计数共享资源、深拷贝资源、转移资源所有权。不能让编译器默认复制资源句柄，否则容易重复释放。



```

class Lock {
public:
    explicit Lock(std::mutex& m) : mutex_(m) {
        mutex_.lock();
    }

    ~Lock() {
        mutex_.unlock();
    }

    Lock(const Lock&) = delete;
    Lock& operator=(const Lock&) = delete;

private:
    std::mutex& mutex_;
};

```

如果资源可以共享，可使用智能指针表达共享语义。

```

class FontHandle {};
void releaseFont(FontHandle*) {}

class Font {
public:
    explicit Font(FontHandle* fh)
        : handle_(fh, releaseFont) {}

private:
    std::shared_ptr<FontHandle> handle_;
};

```

关键是：资源管理类的复制行为必须由设计者决定，而不是交给默认拷贝。

## 条款15：在资源管理类中提供对原始资源的访问

RAII 类封装资源，但有时必须调用旧 API，而旧 API 只接受原始句柄或裸指针。资源管理类应提供显式或隐式访问原始资源的方式。

```
class FontHandle {};  
void changeFontSize(FontHandle*, int);  
  
class Font {  
public:  
    explicit Font(FontHandle* fh) : handle_(fh) {}  
  
    FontHandle* get() const { // 显式转换  
        return handle_.get();  
    }  
  
private:  
    std::shared_ptr<FontHandle> handle_;  
};  
  
void use(Font& f) {  
    changeFontSize(f.get(), 12);  
}
```

显式访问更安全，因为调用处能看出正在穿透封装。隐式转换更方便，但可能被误用。

```
class Font2 {  
public:  
    operator FontHandle*() const {  
        return handle_.get();  
    }  
private:  
    std::shared_ptr<FontHandle> handle_;  
};
```

实际设计中优先提供 `get()` 这类显式函数，只有非常确定时才使用隐式转换。

## 条款16：成对使用 `new` 和 `delete` 时要采取相同形式

`new` 对应 `delete`，`new[]` 对应 `delete[]`。混用会导致析构次数错误，行为未定义。

```
std::string* s1 = new std::string;
delete s1; // 正确

std::string* s2 = new std::string[10];
delete[] s2; // 正确
```

错误示例：

```
std::string* array = new std::string[10];
// delete array; // 错误：只按单个对象删除
```

如果使用类型别名隐藏了数组，更容易误删。

```
using AddressLines = std::string[4];

std::string* pal = new AddressLines;
delete[] pal; // 必须使用 delete[]
```

为了减少这种风险，现代 C++ 应优先使用 `std::vector`、`std::array`、`std::string`、`std::unique_ptr<T[]>` 等容器或智能指针。

## 条款17：以独立语句将 newed 对象置入智能指针

如果在一个函数调用表达式中同时创建智能指针和执行其他参数求值，C++17 之前编译器可能在 `new` 和智能指针构造之间先求值另一个参数。一旦另一个参数抛异常，刚 `new` 出来的对象可能泄漏。C++17 起函数实参之间不再交错求值，但实参顺序仍不固定；本条的实践建议仍然有价值：资源一取得，就立刻交给资源管理对象。

```
class Widget {};  
  
int priority();  
void processWidget(std::shared_ptr<Widget> pw, int priority);  
  
void bad() {  
    // processWidget(std::shared_ptr<Widget>(new Widget), priority());  
    // 如果 priority() 在 new Widget 后、shared_ptr 构造前抛异常，会泄漏  
}
```

应使用独立语句先把资源放进智能指针。

```
void good() {  
    std::shared_ptr<Widget> pw(new Widget);  
    processWidget(pw, priority());  
}
```

现代 C++ 更推荐 `std::make_shared` 或 `std::make_unique`，它们能把对象创建和智能指针管理结合起来，既简洁也更不容易写出裸 `new` 的缝隙。

```
void modern() {  
    processWidget(std::make_shared<Widget>(), priority());  
}
```

## 条款18：让接口容易被正确使用，不易被误用

好接口应该把常见错误变成编译错误，或者让错误很难写出来。可用类型系统区分含义相近的参数，用工厂函数限制创建方式，用智能指针表达所有权。

```
struct Day {
    explicit Day(int d) : value(d) {}
    int value;
};

struct Month {
    explicit Month(int m) : value(m) {}
    int value;
};

struct Year {
    explicit Year(int y) : value(y) {}
    int value;
};

class Date {
public:
    Date(Month m, Day d, Year y)
        : month_(m.value), day_(d.value), year_(y.value) {}

private:
    int month_;
    int day_;
    int year_;
};

Date d(Month(3), Day(30), Year(2026));
// Date bad(Day(30), Month(3), Year(2026)); // 编译错误
```

接口还应尽量与内置类型和标准库习惯一致，例如赋值返回 `*this`，容器提供 `size()`，资源由智能指针返回。

## 条款19：设计 class 犹如设计 type

设计类就是设计一种新类型。要考虑对象如何创建、销毁、复制、赋值，值传递是否合理，合法值是什么，继承关系如何，是否需要类型转换，哪些操作应是成员或非成员。

```
class Rational {
public:
    Rational(int numerator = 0, int denominator = 1)
        : n_(numerator), d_(denominator) {
        if (d_ == 0) {
            throw std::invalid_argument("denominator is zero");
        }
    }

    int numerator() const { return n_; }
    int denominator() const { return d_; }

private:
    int n_;
    int d_;
};
```

这个类的设计问题包括：分母为 0 如何处理，是否自动约分，是否允许 `int` 隐式转换成 `Rational`，算术运算符是否为非成员函数，是否支持异常安全。类设计不是只把数据和函数放在一起，而是为用户定义一套稳定、清晰、难误用的规则。

## 条款20：宁以 pass-by-reference-to-const 替换 pass-by-value

按值传递会复制对象，可能成本高，还可能发生对象切割。对多数自定义类型，传参优先使用 `const T&`。

```

class Person {
public:
    std::string name() const { return name_; }
private:
    std::string name_;
};

class Student : public Person {
public:
    std::string school() const { return school_; }
private:
    std::string school_;
};

void printName(const Person& p) { // 避免复制，也避免切割
    std::cout << p.name();
}

Student s;
printName(s);

```

按值传递派生类给基类参数会切掉派生类部分。

```

void printNameBad(Person p) { // Student 传入后只剩 Person 子对象
    std::cout << p.name();
}

```

例外是内置类型、迭代器、函数对象等小型对象，它们按值传递通常更合适。

## 条款21：必须返回对象时，别妄想返回其 reference

不要返回局部对象、堆上临时对象或静态对象的引用来逃避返回值开销。局部对象会悬空，堆对象会造成释放责任不清，静态对象会造成共享状态错误。

```
class Rational {
public:
    Rational(int n = 0, int d = 1) : n_(n), d_(d) {}
private:
    int n_;
    int d_;
};

const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(); // 正确: 返回新对象
}
```

错误示例:

```
const Rational& badMultiply(const Rational& lhs, const Rational& rhs) {
    Rational result;
    return result; // 错误: 返回局部对象引用
}
```

现代编译器会通过返回值优化、移动语义等手段减少返回对象成本。语义上需要新对象时，就返回对象本身。

## 条款22：将成员变量声明为 private

数据成员应声明为 `private`，通过函数提供访问。这样可以保持封装、控制不变量、保留修改实现的自由。



```

class SpeedDataCollection {
public:
    void addValue(int speed) {
        speeds_.push_back(speed);
    }

    double averageSoFar() const {
        if (speeds_.empty()) {
            return 0.0;
        }
        int sum = std::accumulate(speeds_.begin(), speeds_.end(), 0);
        return static_cast<double>(sum) / speeds_.size();
    }

private:
    std::vector<int> speeds_;
};

```

如果成员变量是 `public`，用户会依赖它的存在、类型和含义，之后几乎无法改成缓存、延迟计算、线程安全实现。`protected` 数据成员也不理想，因为派生类也会形成依赖，同样破坏封装。

## 条款23：宁以 non-member、non-friend 替换 member 函数

如果一个函数可以通过类的公开接口完成工作，就不应为了方便写成成员函数或友元。非成员、非友元函数能减少对私有实现的依赖，提高封装性。

```

class WebBrowser {
public:
    void clearCache() {}
    void clearHistory() {}
    void removeCookies() {}
};

void clearEverything(WebBrowser& browser) { // 非成员、非友元
    browser.clearCache();
    browser.clearHistory();
    browser.removeCookies();
}

```

这样 `clearEverything` 不需要访问私有成员，类本身也不用承担额外接口。若函数很多，可放进同一个命名空间，形成扩展接口。

```

namespace WebBrowserStuff {
    void clearEverything(WebBrowser& browser);
    void saveBookmarks(WebBrowser& browser);
}

```

成员函数越少，类的封装边界越小，后续修改内部实现越自由。

## 条款24：若所有参数皆需类型转换，请为此采用 non-member 函数

成员函数的左侧对象，也就是 `this`，不会像普通参数一样参与隐式类型转换。因此如果一个运算符需要左右两边都支持类型转换，应写成非成员函数。

```

class Rational {
public:
    Rational(int numerator = 0, int denominator = 1)
        : n_(numerator), d_(denominator) {}

    int numerator() const { return n_; }
    int denominator() const { return d_; }

private:
    int n_;
    int d_;
};

const Rational operator*(const Rational& lhs, const Rational& rhs) {
    return Rational(lhs.numerator() * rhs.numerator(),
                    lhs.denominator() * rhs.denominator());
}

Rational oneHalf(1, 2);
Rational result1 = oneHalf * 2; // 2 转成 Rational
Rational result2 = 2 * oneHalf; // 左侧 2 也能转换

```

如果 `operator*` 是成员函数，则 `oneHalf * 2` 可行，而 `2 * oneHalf` 不可行，因为 `2` 没有成员函数 `operator*`。

## 条款25：考虑写出一个不抛异常的 swap 函数

`swap` 对异常安全和高效赋值非常重要。对于使用 `plmpl` 或拥有大量资源的类，应提供高效且不抛异常的 `swap`，并让泛型代码通过非限定调用找到它。

```

class WidgetImpl {
public:
    std::vector<int> data;
};

class Widget {
public:
    Widget() : pImpl_(new WidgetImpl) {}

    void swap(Widget& other) noexcept {
        using std::swap;
        swap(pImpl_, other.pImpl_);
    }

private:
    std::shared_ptr<WidgetImpl> pImpl_;
};

void swap(Widget& a, Widget& b) noexcept {
    a.swap(b);
}

```

使用时写 `using std::swap; swap(a, b);`，这样标准类型走 `std::swap`，自定义类型可通过参数相关查找找到同命名空间下的专属版本。不要为了自定义类型向命名空间 `std` 添加新的重载函数；只有标准允许的全特化才可放入 `std`，普通项目里更推荐提供同命名空间的非成员 `swap`。

```

template <typename T>
void doSomething(T& a, T& b) {
    using std::swap;
    swap(a, b);
}

```

## 条款26：尽可能延后变量定义式的出现时间

变量定义得越早，越可能在未使用前就承担构造和析构成本，也会扩大作用域、增加误用可能。应等到知道需要它并能给出初值时再定义。

```
std::string encryptPassword(const std::string& password) {
    if (password.length() < 8) {
        throw std::logic_error("password too short");
    }

    std::string encrypted(password); // 通过检查后才构造
    // encrypt encrypted
    return encrypted;
}
```

循环中变量定义在循环内还是循环外，要看成本和语义。若每次迭代都需要独立状态，放在循环内更清晰。

```
for (int i = 0; i < n; ++i) {
    Widget w(args); // 每轮需要不同对象时放这里
    // use w
}
```

延后定义通常也能更容易做到“定义即初始化”。

## 条款27：尽量少做转型

转型会绕过类型系统，隐藏设计问题，也可能生成额外代码。C++ 风格转型应优先于 C 风格转型，因为意图更明确：`const_cast` 去除常量性，`dynamic_cast` 安全向下转型，`reinterpret_cast` 低级重解释，`static_cast` 常规显式转换。

```

class Base {
public:
    virtual ~Base() = default;
};

class Derived : public Base {
public:
    void special() {}
};

void f(Base* p) {
    if (Derived* d = dynamic_cast<Derived*>(p)) {
        d->special();
    }
}

```

不要为了调用派生类函数而到处 `dynamic_cast`。如果行为属于多态体系，优先放进虚函数。

```

class Window {
public:
    virtual ~Window() = default;
    virtual void blink() {}
};

void blinkAll(std::vector<std::unique_ptr<Window>>& windows) {
    for (auto& w : windows) {
        w->blink(); // 用多态代替转型
    }
}

```

## 条款28：避免返回 handles 指向对象内部成分

不要返回指针、引用、迭代器等 handle 指向对象内部数据，否则会破坏封装，可能让外部修改内部状态，也可能在对象销毁后留下悬空 handle。

```

class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
    void setX(int x) { x_ = x; }
private:
    int x_;
    int y_;
};

class Rectangle {
public:
    const Point& upperLeft() const { // 仍有悬空风险
        return ul_;
    }

private:
    Point ul_;
};

```

即使返回 `const` 引用，调用者不能修改，但仍可能保存引用并在对象销毁后使用。更安全的做法是返回值对象，或提供受控操作。

```

class SafeRectangle {
public:
    Point upperLeft() const { // 返回副本
        return ul_;
    }

private:
    Point ul_{0, 0};
};

```

## 条款29：为异常安全而努力是值得的

异常安全函数应满足三个保证之一：基本保证，异常后对象仍有效；强烈保证，异常后程序状态不变；不抛异常保证，承诺不抛。设计时应尽量让资源由对象管理，并用 `copy-and-swap` 提供强烈保证。

```

class Image {};

class PrettyMenu {
public:
    void changeBackground(std::shared_ptr<Image> newImage) {
        bgImage_ = std::move(newImage); // shared_ptr 赋值异常安全
        ++imageChanges_;
    }

private:
    std::shared_ptr<Image> bgImage_;
    int imageChanges_ = 0;
};

```

copy-and-swap 示例：

```

class Widget {
public:
    Widget& operator=(Widget rhs) { // 先复制，失败则当前对象不变
        swap(rhs);
        return *this;
    }

    void swap(Widget& other) noexcept {
        using std::swap;
        swap(data_, other.data_);
    }

private:
    std::vector<int> data_;
};

```

异常安全不是事后补丁，而是接口、资源管理和状态更新顺序共同决定的。



## 条款30：透彻了解 inlining 的里里外外

`inline` 是对编译器的请求，不是命令。内联可减少函数调用开销，也可能导致目标码膨胀、缓存命中下降、编译依赖增加。短小、频繁调用的函数适合内联，复杂函数、递归函数、取地址函数通常不适合。

```
class Person {
public:
    int age() const { return age_; } // 类内定义，隐式 inline

private:
    int age_ = 0;
};
```

模板函数通常放在头文件中，容易被内联，但这也意味着实现变化会导致使用者重新编译。

```
template <typename T>
inline const T& minValue(const T& a, const T& b) {
    return b < a ? b : a;
}
```

不要把 `inline` 当性能开关。性能关键代码应以测量为准，接口稳定性和编译成本也要考虑。

## 条款31：将文件间的编译依存关系降至最低

头文件包含越多，改动传播越广，编译越慢。应尽量用前置声明、接口类、`plmpl` 等技术减少头文件对实现细节的依赖。

```
// Person.h
#include <memory>
#include <string>

class Person {
public:
    Person(std::string name, int age);
    std::string name() const;
    int age() const;

private:
    class Impl;
    std::shared_ptr<Impl> pImpl_;
};
```

```
// Person.cpp
#include "Person.h"

class Person::Impl {
public:
    std::string name;
    int age = 0;
};

Person::Person(std::string name, int age)
    : pImpl_(std::make_shared<Impl>()) {
    pImpl_->name = std::move(name);
    pImpl_->age = age;
}
```

这样用户只依赖 `Person` 的接口，不依赖 `Impl` 的成员布局。实现变更时，使用者不一定需要重新编译。

## 条款32：确定你的 public 继承塑模出 is-a 关系

`public` 继承表示“派生类对象是一个基类对象”。凡是能对基类成立的行为，也必须能对派生类成立。若不满足，就不该使用 `public` 继承。

经典反例是正方形继承矩形。

```
class Rectangle {
public:
    virtual void setWidth(int w) { width_ = w; }
    virtual void setHeight(int h) { height_ = h; }
    int width() const { return width_; }
    int height() const { return height_; }

private:
    int width_ = 0;
    int height_ = 0;
};

void makeBigger(Rectangle& r) {
    int oldHeight = r.height();
    r.setWidth(r.width() + 10);
    // 对普通矩形，高度不应改变
}
```

正方形设置宽度时必须同步高度，不符合矩形接口承诺。因此数学上的“正方形是矩形”不等于软件接口中的 is-a。

## 条款33：避免遮掩继承而来的名称

派生类中声明同名函数会遮掩基类所有同名函数，即使参数不同。需要把基类重载函数引入派生类作用域。

```

class Base {
public:
    virtual void mf1() {}
    virtual void mf1(int) {}
    void mf2() {}
};

class Derived : public Base {
public:
    using Base::mf1;
    using Base::mf2;

    void mf1() override {}
    void mf2(int) {}
};

Derived d;
d.mf1();
d.mf1(10); // 如果没有 using Base::mf1, 这里会找不到

```

如果只想暴露基类某个函数，也可写转交函数。

```

class PrivateDerived : private Base {
public:
    void mf1() {
        Base::mf1();
    }
};

```

## 条款34：区分接口继承和实现继承

纯虚函数表示只继承接口；普通虚函数表示继承接口和默认实现；非虚函数表示继承接口和强制实现。三者含义不同，应根据设计意图选择。

```

class Shape {
public:
    virtual void draw() const = 0;          // 只继承接口
    virtual void error(const std::string& msg) { // 接口 + 默认实现
        std::cerr << msg;
    }
    int objectId() const { return id_; } // 接口 + 固定实现

private:
    int id_ = 0;
};

```

如果派生类必须自己实现，就用纯虚函数。如果大多数派生类共享默认行为，但允许改写，用虚函数。如果行为不应改变，用非虚函数。不要把“方便复用代码”误当成“应该提供默认虚函数”。

## 条款35：考虑 virtual 函数以外的其他选择

多态行为不一定只能用虚函数。可用非虚接口 NVI、函数指针、`std::function`、策略模式等替代。NVI 让公共非虚函数负责固定流程，私有虚函数负责可变步骤。

```

class GameCharacter {
public:
    int healthValue() const { // 非虚接口
        int retVal = doHealthValue();
        return retVal;
    }

private:
    virtual int doHealthValue() const { // 可由派生类改写
        return 100;
    }
};

```

策略对象能在运行期替换算法。

```
class GameCharacter2 {
public:
    using HealthCalc = std::function<int(const GameCharacter2&)>;

    explicit GameCharacter2(HealthCalc calc) : calc_(std::move(calc)) {}

    int healthValue() const {
        return calc_(*this);
    }

private:
    HealthCalc calc_;
};
```

这些方案能降低继承耦合，让算法组合更灵活。

## 条款36：绝不重新定义继承而来的 non-virtual 函数

非虚函数表示基类规定的固定行为。派生类重新定义同名非虚函数时，通过基类指针和派生类对象调用会得到不同结果，破坏 is-a 关系。

```

class B {
public:
    void mf() {
        std::cout << "B";
    }
};

class D : public B {
public:
    void mf() {
        std::cout << "D";
    }
};

D x;
B* pb = &x;
D* pd = &x;

pb->mf(); // 输出 B
pd->mf(); // 输出 D

```

同一个对象因静态类型不同表现不同，非常危险。如果希望派生类改变行为，基类函数应设计为 `virtual`；如果不希望改变，派生类就不应重定义。

## 条款37：绝不重新定义继承而来的缺省参数值

虚函数是动态绑定，但默认参数是静态绑定。若派生类重写虚函数并改变默认参数，通过基类指针调用时会使用基类默认参数，却执行派生类函数。

```

class Shape {
public:
    enum ShapeColor { Red, Green, Blue };
    virtual void draw(ShapeColor color = Red) const = 0;
};

class Rectangle : public Shape {
public:
    void draw(ShapeColor color = Green) const override {}
};

Rectangle r;
Shape* ps = &r;
ps->draw(); // 调用 Rectangle::draw, 但默认参数是 Shape::Red

```

解决方法是不要在派生类中重新定义不同默认值。更稳妥的是使用 NVI，把默认参数放在非虚接口上。

```

class Shape2 {
public:
    void draw(ShapeColor color = Red) const {
        doDraw(color);
    }

private:
    virtual void doDraw(ShapeColor color) const = 0;
};

```

## 条款38：通过复合塑模出 has-a 或根据某物实现出

复合表示一个对象包含另一个对象。应用领域中常表示 has-a，例如人有地址；实现领域中常表示 is-implemented-in-terms-of，例如集合可用链表实现。



```

class Address {};

class Person {
public:
    Address address() const { return address_; }

private:
    Address address_; // Person has an Address
};

```

如果只是想复用实现，复合通常比继承更合适。

```

template <typename T>
class Set {
public:
    bool member(const T& item) const {
        return std::find(rep_.begin(), rep_.end(), item) != rep_.end();
    }

    void insert(const T& item) {
        if (!member(item)) {
            rep_.push_back(item);
        }
    }

private:
    std::list<T> rep_; // 根据 list 实现 Set，但 Set 不是 list
};

```

不要因为“用到了某类的功能”就继承它。

## 条款39：明智而审慎地使用 private 继承

`private` 继承表示 is-implemented-in-terms-of，不表示 is-a。它和复合都能复用实现，但复合通常更清晰。只有在需要访问基类 `protected` 成员、重写虚函数，或利用空基类优化时，`private` 继承才更有理由。

```

class Timer {
public:
    virtual void onTick() const {}
    virtual ~Timer() = default;
};

class Widget : private Timer {
private:
    void onTick() const override {
        // Widget 用 Timer 的机制实现定时行为
    }
};

```

如果不需要重写虚函数，用复合更好。

```

class Widget2 {
private:
    Timer timer_;
};

```

判断标准：能用复合表达清楚，就优先复合；`private` 继承只用于确有技术需要的场景。

## 条款40：明智而审慎地使用多重继承

多重继承容易带来二义性、菱形继承、虚继承成本和复杂初始化顺序。能不用时尽量不用；确实使用时，要让每个基类职责清楚。

```

class BorrowableItem {
public:
    void checkOut() {}
};

class ElectronicGadget {
private:
    bool checkOut() const { return true; }
};

class MP3Player : public BorrowableItem, public ElectronicGadget {};

MP3Player mp;
// mp.checkOut(); // 二义性或访问问题
mp.BorrowableItem::checkOut();

```

合理场景之一是“一个 public 接口基类 + 一个 private 实现辅助基类”。

```

class IPerson {
public:
    virtual ~IPerson() = default;
    virtual std::string name() const = 0;
};

class PersonInfo {
protected:
    std::string theName() const { return "name"; }
};

class CPerson : public IPerson, private PersonInfo {
public:
    std::string name() const override {
        return theName();
    }
};

```

## 条款41：了解隐式接口和编译期多态

面向对象的接口通常是显式的，由函数签名和虚函数构成；模板的接口是隐式的，由模板代码中对类型的表达式要求构成。虚函数多态发生在运行期，模板多态发生在编译期。

```
template <typename T>
void doProcessing(T& w) {
    if (w.size() > 10 && w != T()) {
        T temp(w);
        temp.normalize();
        temp.swap(w);
    }
}
```

这段模板隐式要求 `T` 支持：

`size()`、`operator>`、`operator!=`、默认构造、拷贝构造、`normalize()`、`swap()`。

满足这些表达式的类型都可以使用该模板，而不必继承某个基类。

```
class Image {
public:
    std::size_t size() const { return 20; }
    void normalize() {}
    void swap(Image&) {}
};

bool operator!=(const Image&, const Image&) {
    return true;
}
```

模板错误常在实例化时出现，本质就是隐式接口不满足。

## 条款42：了解 typename 的双重意义

在模板参数列表中，`typename` 和 `class` 基本等价。但在模板内部引用依赖类型名称时，必须用 `typename` 告诉编译器这是一个类型。

```
template <typename C>
void print2nd(const C& container) {
    if (container.size() >= 2) {
        typename C::const_iterator iter(container.begin());
        ++iter;
        std::cout << *iter;
    }
}
```

没有 `typename` 时，编译器不知道 `C::const_iterator` 是类型还是静态成员。例外是基类列表和成员初始化列表中不允许这样使用。

```
template <typename T>
class Derived : public Base<T>::Nested { // 基类列表中不用 typename
public:
    explicit Derived(int x)
        : Base<T>::Nested(x) {}          // 初始化列表中不用 typename
};
```

规则简记：模板中依赖于模板参数的嵌套类型名称，通常要加 `typename`。

## 条款43：学习处理模板化基类内的名称

模板派生类不会自动在模板基类中查找名称，因为基类依赖模板参数，编译器不知道特化后基类里是否有该名称。需要用 `this->`、`using` 或明确基类限定。

```

template <typename Company>
class MsgSender {
public:
    void sendClear(const std::string& msg) {}
};

template <typename Company>
class LoggingMsgSender : public MsgSender<Company> {
public:
    void sendClearMsg(const std::string& msg) {
        this->sendClear(msg); // 告诉编译器到依赖基类中找
    }
};

```

也可以使用 `using`。

```

template <typename Company>
class LoggingMsgSender2 : public MsgSender<Company> {
public:
    using MsgSender<Company>::sendClear;

    void sendClearMsg(const std::string& msg) {
        sendClear(msg);
    }
};

```

明确限定 `MsgSender<Company>::sendClear(msg)` 也可行，但如果它是虚函数，会抑制虚分派，需谨慎。

## 条款44：将与参数无关的代码抽离 templates

模板会为不同类型或不同非类型参数生成多份代码。若模板中有与参数无关的逻辑，应提取到非模板函数或较少参数的基类中，减少代码膨胀。

```
template <typename T, std::size_t n>
class SquareMatrix {
public:
    void invert() {
        // 针对每个 T 和 n 都生成一份，可能膨胀
    }
};
```

可把与矩阵大小无关的实现抽到基类函数中。

```
template <typename T>
class SquareMatrixBase {
protected:
    void invert(std::size_t matrixSize) {
        // 与 n 无关的实现放这里
    }
};

template <typename T, std::size_t n>
class SquareMatrix : private SquareMatrixBase<T> {
public:
    void invert() {
        SquareMatrixBase<T>::invert(n);
    }
};
```

类型参数也会造成膨胀。例如 `vector<int*>`、`vector<void*>` 的底层指针操作可能相同，设计库时应尽量共享实现。

## 条款45：运用成员函数模板接受所有兼容类型

智能指针、迭代器等模板类常需要从兼容类型转换，例如 `shared_ptr<Derived>` 转成 `shared_ptr<Base>`。普通拷贝构造只接受同一类型，成员函数模板可接受兼容类型。

```

class Top {};
class Middle : public Top {};
class Bottom : public Middle {};

template <typename T>
class SmartPtr {
public:
    explicit SmartPtr(T* realPtr) : ptr_(realPtr) {}

    template <typename U>
    SmartPtr(const SmartPtr<U>& other)
        : ptr_(other.get()) {}

    T* get() const { return ptr_; }

private:
    T* ptr_;
};

SmartPtr<Bottom> pb(new Bottom);
SmartPtr<Top> pt(pb); // 通过成员函数模板接受兼容类型

```

成员函数模板不会阻止编译器生成普通拷贝构造函数和拷贝赋值运算符。需要时仍要声明普通版本。

## 条款46：需要类型转换时请为模板定义非成员函数

模板类中的成员函数模板在某些类型转换场景中推导不出参数。若运算需要隐式类型转换，应在类模板内部声明并定义友元非成员函数，使它随类模板实例化。



```

template <typename T>
class Rational {
public:
    Rational(const T& numerator = 0, const T& denominator = 1)
        : n_(numerator), d_(denominator) {}

    const T numerator() const { return n_; }
    const T denominator() const { return d_; }

    friend const Rational operator*(const Rational& lhs,
                                    const Rational& rhs) {
        return Rational(lhs.n_ * rhs.n_, lhs.d_ * rhs.d_);
    }

private:
    T n_;
    T d_;
};

Rational<int> oneHalf(1, 2);
Rational<int> result = oneHalf * 2; // 2 可转换为 Rational<int>

```

这里友元函数不是函数模板，而是随着 `Rational<T>` 生成的普通函数，因此能参与隐式转换。

## 条款47：请使用 traits classes 表现类型信息

traits class 用类型表示类型信息，让算法在编译期根据类型特性选择实现。STL 的 `iterator_traits` 就是典型例子。

```

template <typename IterT>
void advanceImpl(IterT& iter, int d, std::random_access_iterator_tag) {
    iter += d;
}

template <typename IterT>
void advanceImpl(IterT& iter, int d, std::bidirectional_iterator_tag) {
    if (d >= 0) {
        while (d-- > 0) ++iter;
    } else {
        while (d++ < 0) --iter;
    }
}

template <typename IterT>
void myAdvance(IterT& iter, int d) {
    typename std::iterator_traits<IterT>::iterator_category category;
    advanceImpl(iter, d, category);
}

```

`vector<int>::iterator` 是随机访问迭代器，走 `+=`；`list<int>::iterator` 是双向迭代器，走循环。选择发生在编译期，避免运行期开销。

## 条款48：认识 template 元编程

模板元编程是在编译期执行计算和选择逻辑。它能把一部分运行期工作提前到编译期，也能根据类型生成不同代码，但会增加复杂度和编译时间。

```

template <unsigned n>
struct Factorial {
    enum { value = n * Factorial<n - 1>::value };
};

template <>
struct Factorial<0> {
    enum { value = 1 };
};

int x = Factorial<5>::value; // 编译期得到 120

```

现代 C++ 可用 `constexpr` 和标准 type traits 写得更清楚。

```

constexpr unsigned factorial(unsigned n) {
    return n == 0 ? 1 : n * factorial(n - 1);
}

static_assert(factorial(5) == 120);

```

模板元编程适合泛型库、类型萃取、编译期分派，不应滥用于普通业务逻辑。

## 条款49：了解 new-handler 的行为

当 `operator new` 分配失败时，会调用当前的 `new-handler`。`new-handler` 可以释放内存、安装另一个 handler、卸除 handler、抛出 `bad_alloc`，或终止程序。

```
void outOfMem() {  
    std::cerr << "Unable to allocate memory\n";  
    std::abort();  
}  
  
int main() {  
    std::set_new_handler(outOfMem);  
    int* p = new int[1000000000];  
    delete[] p;  
}
```

类也可以拥有自己的 new-handler，通常做法是在类的 `operator new` 中临时设置全局 handler，再调用全局 `operator new`。临时替换全局 handler 必须异常安全，否则 `::operator new` 抛异常时，全局 handler 无法恢复。

```

class Widget {
public:
    static std::new_handler set_new_handler(std::new_handler p) noexcept
    {
        std::new_handler old = currentHandler_;
        currentHandler_ = p;
        return old;
    }

    static void* operator new(std::size_t size) {
        NewHandlerHolder h(std::set_new_handler(currentHandler_));
        return ::operator new(size);
    }

private:
    class NewHandlerHolder {
    public:
        explicit NewHandlerHolder(std::new_handler nh) noexcept
            : handler_(nh) {}

        ~NewHandlerHolder() {
            std::set_new_handler(handler_);
        }

        NewHandlerHolder(const NewHandlerHolder&) = delete;
        NewHandlerHolder& operator=(const NewHandlerHolder&) = delete;

    private:
        std::new_handler handler_;
    };

    static std::new_handler currentHandler_;
};

```

## 条款50：了解 new 和 delete 的合理替换时机

替换 `operator new` 和 `operator delete` 的目的通常是：检测使用错误、改善性能、收集统计信息、支持内存池、满足特殊对齐需求。不要为了“看起来

高级”而替换。

```
class PoolObject {
public:
    static void* operator new(std::size_t size) {
        // 可接入内存池；这里简化为调用全局版本
        return ::operator new(size);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }
};
```

调试时可以在分配块前后加边界标记，检查越界写。但自定义分配器很容易破坏对齐、线程安全、异常约定和性能假设。实际项目中，优先考虑标准库容器的 allocator、成熟内存池或平台工具。

## 条款51：编写 new 和 delete 时需固守常规

自定义 `operator new` 必须返回正确对齐的内存；失败时应调用 `new-handler`，无法处理再抛 `std::bad_alloc`；大小为 0 的申请也应返回合法唯一指针。`operator delete` 必须能安全处理空指针。

```

class Base {
public:
    static void* operator new(std::size_t size) {
        if (size != sizeof(Base)) {
            return ::operator new(size); // 派生类大小不同，转给全局版本
        }

        if (size == 0) {
            size = 1;
        }

        while (true) {
            if (void* p = std::malloc(size)) {
                return p;
            }

            std::new_handler globalHandler = std::get_new_handler();
            if (globalHandler) {
                (*globalHandler)();
            } else {
                throw std::bad_alloc();
            }
        }
    }

    static void operator delete(void* rawMemory) noexcept {
        if (rawMemory == nullptr) {
            return;
        }
        std::free(rawMemory);
    }
};

```

自定义分配函数还要注意继承：基类的 `operator new` 会被派生类继承，但派生类对象大小可能不同。

## 条款52：写了 placement new 也要写 placement delete

如果构造函数在 placement new 分配成功后抛异常，运行期会寻找匹配的 placement delete 来释放内存。若没有匹配版本，内存会泄漏。

```
class Widget {
public:
    static void* operator new(std::size_t size, std::ostream& log) {
        log << "allocating Widget\n";
        return ::operator new(size);
    }

    static void operator delete(void* p, std::ostream& log) noexcept {
        log << "deleting after constructor failure\n";
        ::operator delete(p);
    }

    static void operator delete(void* p) noexcept {
        ::operator delete(p);
    }

    Widget() {
        // 如果这里抛异常，会调用 operator delete(void*, std::ostream&)
    }
};

Widget* pw = new (std::cerr) Widget;
delete pw; // 正常 delete 调用普通 operator delete
```

注意 placement delete 只在构造失败时自动调用；对象构造成功后，普通 delete 不会传入 placement 参数。

## 条款53：不要轻忽编译器的警告

编译器警告常指出真实缺陷，例如未初始化变量、隐藏虚函数、可能的截断、基类析构函数非虚等。应把警告当作需要解释的问题，而不是噪声。



```
class B {
public:
    virtual void f() const {}
};

class D : public B {
public:
    void f() {} // 少了 const, 不是 override, 可能触发警告
};
```

现代 C++ 可用 `override` 把这类问题变成编译错误。

```
class D2 : public B {
public:
    void f() const override {}
};
```

不同编译器警告能力不同，不要因为某个编译器无警告就认为代码正确。跨编译器、提高警告等级、配合静态分析，能更早暴露问题。

## 条款54：让自己熟悉包括 TR1 在内的标准程序库

第三版写作时 TR1 是标准库扩展的重要来源，包含智能指针、函数对象包装器、正则表达式、哈希容器、随机数等。现代 C++ 中，许多 TR1 内容已经进入标准库。

```

#include <functional>
#include <memory>
#include <unordered_map>

class Widget {};

void process(std::shared_ptr<Widget>) {}

int main() {
    auto pw = std::make_shared<Widget>();
    process(pw);

    std::unordered_map<std::string, int> score;
    score["Alice"] = 95;

    std::function<int(int, int)> add = [](int a, int b) {
        return a + b;
    };
}

```

熟悉标准库能减少手写代码：容器管理内存，算法表达意图，智能指针管理资源，函数包装器统一可调用对象。能用标准库解决的问题，通常不要先造轮子。

## 条款55：让自己熟悉 Boost

Boost 是高质量 C++ 库集合，很多组件曾影响或进入标准库。第三版强调 Boost 的价值：它提供大量经过实践检验的组件，也展示了泛型编程和库设计技巧。

```
// 现代 C++ 中，许多当年的 Boost/TR1 功能已有标准库版本：
#include <optional>
#include <variant>
#include <filesystem>

std::optional<int> parseNumber(const std::string& s) {
    try {
        return std::stoi(s);
    } catch (...) {
        return std::nullopt;
    }
}

int main() {
    std::variant<int, std::string> value = "ok";
    std::filesystem::path p = "data.txt";
}
```

如果项目使用较旧 C++ 标准，Boost 仍可能提供标准库缺失能力；即使不用 Boost，阅读其接口设计也能学习 RAII、泛型编程、类型萃取、可移植性封装等技巧。现代项目中应优先检查标准库是否已有对应组件，再决定是否引入 Boost。